AUGUST HACKATHON WORKBOOK

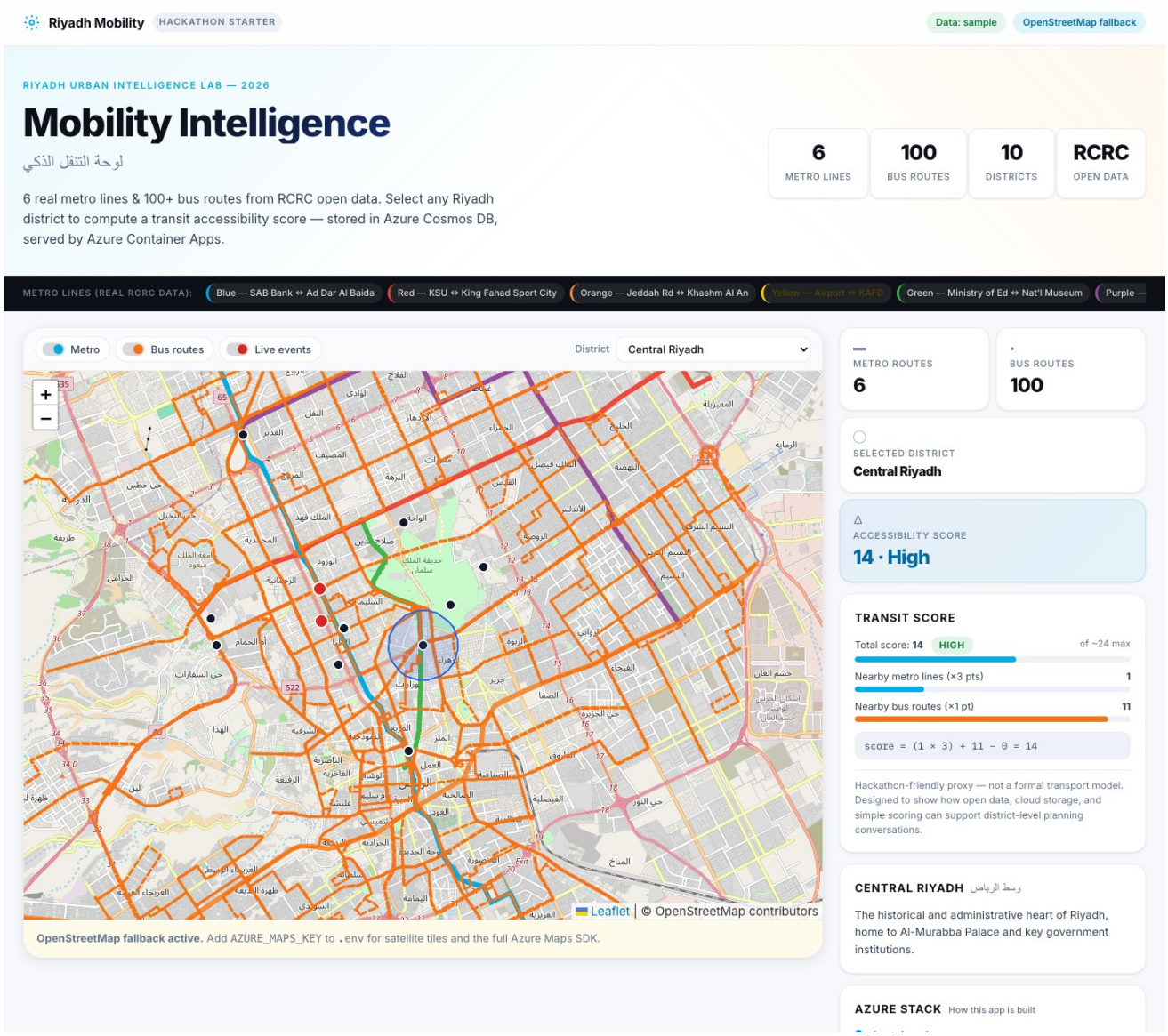
Riyadh Mobility Intelligence Starter Kit

Build and Deployment Workbook

A polished, local-first build path for teams adapting mobility intelligence, district scoring, and Azure-backed urban development prototypes.

Document targets

Run locally Sample data, map fallback, district selector, score panel local	**Explain the system** FastAPI, JavaScript, data fallback, Azure services system
Deploy credibly Container Apps, Maps, Blob, Cosmos, App Insights cloud	**Adapt by track** Technology, People, Sustainability, Culture reuse



Transformational Technology | Prosperous People | Azure-backed path | Sample-first fallback

Prepared for atomcamp Arabia and Microsoft collaboration

How to Use This Guide

A practical build path for teams who need to understand, run, inspect, deploy, and adapt the Riyadh Mobility Intelligence starter kit with minimal help.

Quick access	Link / location
GitHub repo	https://github.com/Esturban/microsoft-hackathon-riyadh-mobility
Local app	http://127.0.0.1:8000
Deployed app	Paste WEB_APP_URL from azd output here
Resource index	Appendix: Azure docs, deployment notes, project references

Builder playbook

01 Understand app Read what the dashboard, layers, score panel, and fallback chain are doing.	**02 Run locally** Create the Python environment, start FastAPI, and wait for the slow initial load.
03 Inspect API Check /health, /api/data-status, routes, districts, and score responses.	**04 Select district** Use Riyadh North or Al Olaya and confirm the score panel updates.
05 Deploy Azure Use az login, azd auth login, and scripts/deploy_azure.sh when ready.	**06 Adapt track** Keep the scaffold; replace data, scoring, layers, and Azure services by track.

Local-first principle: Get the dashboard working from bundled sample data before adding cloud credentials, Blob Storage, Cosmos DB, or a deployed app URL.

Executive Build Brief

A local-first, Azure-ready workbook for teams turning open mobility data into a credible urban intelligence prototype.

Proof points

106
route records across metro and bus sample layers
sample data

10
district records available for scoring and demos
districts

9
API endpoints to inspect, extend, and deploy
contracts

7+
Azure services mapped to clear runtime jobs
cloud

What this proves

Working dashboard
A connected map, district context, explainable score, and data fallback.

Starter kit, not final model
The score is deliberately simple; the value is the transparent scaffold.

Build path

Run sample mode
Confirm local app, fallback, route layers, selector, and panel.

Inspect contracts
Review health, data-status, routes, districts, and score.

Connect Azure
Deploy hosting, maps, files, records, monitoring, and diagnostics.

Adapt for track
Replace data and scoring while keeping the shell and deployment path.

Design principle: Keep the first successful path simple: sample data locally, transparent API behavior, then Azure services only when the team is ready to make the prototype cloud-live.

1. What Builders Are Actually Building

A local-first mobility intelligence scaffold: open the dashboard, inspect transport layers, select a district, and explain an access score.

Core experience

Map the network

Show metro and bus layers with fallback tiles when Azure Maps is not configured.

app/static/src/map.js

Select a district

Anchor every demo around Riyadh district records.

/api/districts

Explain the score

Turn nearby metro, bus, and event inputs into a replaceable score.

app/scoring.py

Starter-kit principle

Run without cloud

Bundled sample data keeps the app useful before Azure is ready.

DATA_MODE=sample

Keep contracts stable

The frontend and API use the same contract locally and in Azure.

FastAPI + static frontend

Swap the domain

Replace data, scoring, or overlays while keeping the app shell.

track adaptation

Judge-facing proof

Visible data layers

Show transit coverage, district context, and live event markers.

dashboard

Transparent logic

The score formula is simple enough to explain live.

/api/score

Azure-ready path

The same containerized app can move to Azure when ready.

azure.yaml + infra/

Product thesis: Prove the end-to-end build path first, then let teams make the domain model more ambitious.

2. Urban Challenge Track Mapping

Lead with Transformational Technology, then use the same district-scoring scaffold for Prosperous People and targeted extension tracks.

Primary story

Transformational Technology

Use mobility layers, route visibility, fallback-aware data, APIs, and Azure deployment to prove a working urban intelligence path.

primary fit

Prosperous People

Convert the district score into service access, walkability, 15-minute city coverage, or quality-of-life comparison.

strong secondary

Builder pitch

Show a credible local app first, then explain how the same scaffold can support richer district intelligence.

demo narrative

Extension paths

Sustainable Solutions

Add AQI, heat, emissions, low-carbon route scoring, and clean-corridor recommendations after the core map works.

extension

Culture

Add heritage sites, event-day routing, visitor flows, multilingual notes, and crowd-sensitive access planning.

extension

What to keep

Keep the frontend shell, FastAPI contracts, sample fallback, score panel, and Azure deployment shape.

reuse

Azure emphasis

Maps

Azure Maps supports the cloud map path once keys are configured.

AZURE_MAPS_KEY

Data

Blob Storage and Cosmos DB support processed files and app-shaped records.

Blob + Cosmos

Runtime

Container Apps, App Insights, and Log Analytics make the prototype deployable and observable.

cloud-live

Positioning rule: Do not sell this as a final transport model. Sell it as a credible build path for urban intelligence prototypes.

Services and Tools Matrix

The stack reads best as a set of clear jobs: local app runtime, data and maps, Azure hosting, and diagnostics. Each tool has one reason to exist.

Local app runtime

FastAPI Backend API plus static frontend serving. app/main.py, app/routes.py	**Vanilla JavaScript** Dashboard boot, map controls, panels, and API calls. app/static/src/	**Sample files** Guaranteed fallback so teams can demo without Azure. app/static/sample-data/

Map and cloud data

Azure Maps Cloud map SDK and services when a key is configured. AZURE_MAPS_KEY	**Blob Storage** Processed GeoJSON and JSON files for cloud data mode. upload_to_blob.py	**Cosmos DB** Route, district, and event records for app-shaped data. seed_cosmos.py

Deploy and observe

Container Apps Hosted FastAPI/frontend runtime. azure.yaml, infra/main.bicep	**Container Registry** Container image storage for the deployed app. infra/modules/	**App Insights + Logs** Health, failures, latency, requests, and diagnostics. Azure portal

Infrastructure workflow

Bicep Infrastructure-as-code for reproducible Azure resources. infra/main.bicep	**Azure Developer CLI** Environment provisioning and deploy workflow. azd / deploy script	**Tests** API, data-shape, health, and scoring checks. tests/

3. Architecture and Build Path

The app is designed to work locally first, then graduate into an Azure-backed live path. The app must remain useful even when Azure credentials, remote services, or live data are unavailable.

Flow map

Local-first path

Browser -> FastAPI -> static frontend/API -> bundled sample data -> score panel.

no Azure required

Azure-backed path

Browser -> Container Apps -> FastAPI -> Azure Maps / Blob / Cosmos / App Insights.

cloud-live

Fallback chain

Cosmos when configured -> Blob when configured -> bundled sample files always.

resilient data

Score request flow

User selects district -> frontend calls /api/score -> backend loads data -> panel renders result.

explainable

Editable build-path diagrams

Browser to backend

Browser opens the static dashboard served by FastAPI.

Browser -> FastAPI

Backend to data

FastAPI loads Cosmos, Blob, or bundled sample files through one data-access layer.

API -> data

Data to interface

GeoJSON, route records, events, and score components return to the dashboard.

data -> UI

Fallback order

Use Cosmos when configured, Blob when configured, and sample files always.

Cosmos -> Blob -> sample

Builder move: Start with DATA_MODE=sample, then move to Blob or Cosmos only after the local app is working.

4. Project Structure

Start with the app shell, API, frontend, sample data, deployment files, and tests.

```
riyadh-mobility-intelligence-dashboard/
├── app/                          # FastAPI app, API, frontend, sample data
├── scripts/                      # fetch, normalize, upload, seed, validate
├── infra/                        # Bicep infrastructure modules
├── docs/                         # build guide and supporting notes
├── tests/                        # scoring, API, and data-shape checks
├── azure.yaml                    # Azure Developer CLI project
├── Dockerfile                    # container definition
└── requirements.txt              # Python dependencies
```

Area	Start here	Why it matters
Backend entrypoints	app/main.py, app/routes.py	FastAPI app shell, static serving, API contract
Data loading	app/data_access.py, app/static/sample-data/	sample, Blob, and Cosmos fallback behavior
Score formula	app/scoring.py	transparent scoring logic that teams can adapt
Frontend	app/static/index.html, app/static/src/main.js	page shell, boot sequence, API orchestration
Map behavior	app/static/src/map.js, app/static/src/layers.js	Azure Maps path, local fallback, overlays
Deployment	azure.yaml, infra/main.bicep, scripts/deploy_azure.sh	cloud-live path
Tests	tests/	API, data-shape, health, and scoring checks

Where the Data Comes From

The bundled sample data is derived from Riyadh Commission for Riyadh City open geospatial datasets. The files live under app/static/sample-data/ and keep the app useful even when no cloud services are configured.

File	Contents	Records
riyadh_metro_lines_sample.geojson	Metro line features with names, colors, and geometry	6

riyadh_bus_routes_sample.geojson	Bus route features with route IDs and geometry	100
district_centers_sample.geojson	Riyadh district center points with English and Arabic names	10
mock_live_events_sample.json	Sample delay or incident markers	variable

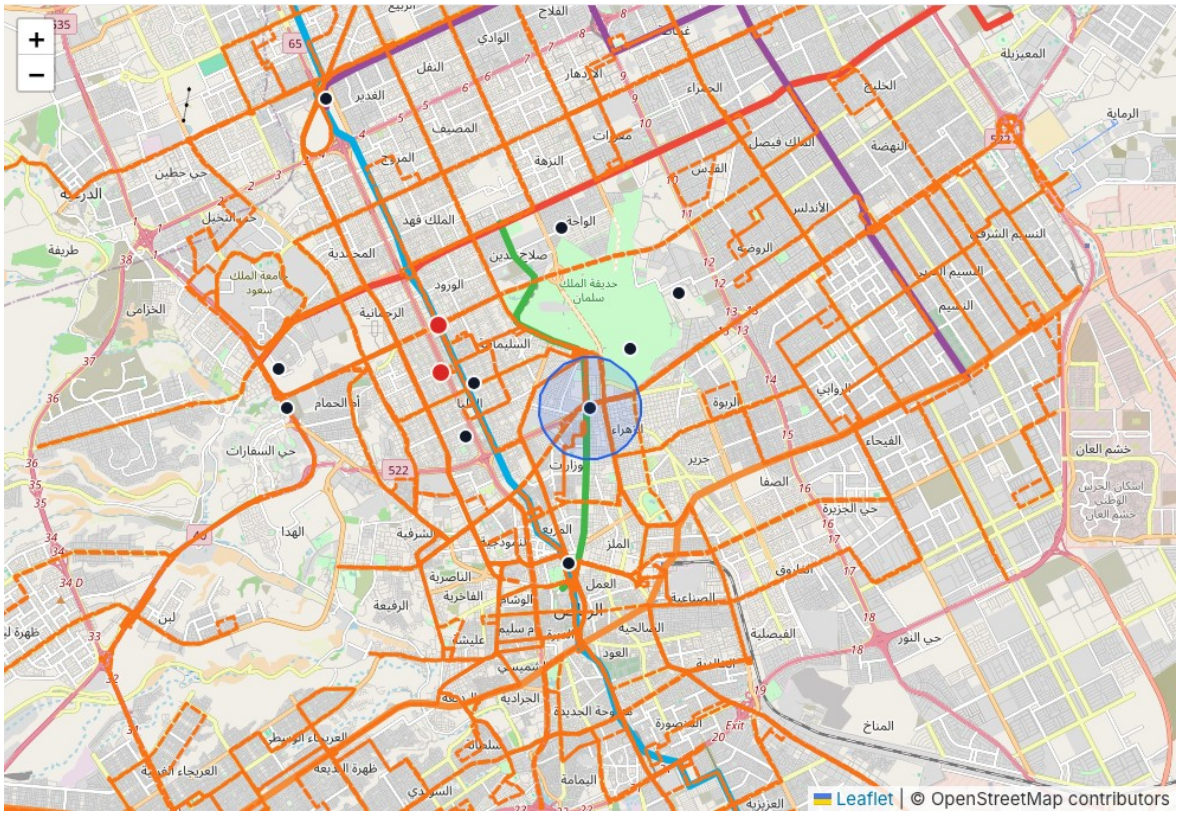
5. Run Locally

Local setup is deliberately direct. The goal is to get one useful screen running before adding cloud services.

```
git clone https://github.com/Esturban/microsoft-hackathon-riyadh-mobility.git
cd microsoft-hackathon-riyadh-mobility
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
```

```
DATA_MODE=sample
python -m uvicorn app.main:app --reload
http://127.0.0.1:8000
```

Slow-load warning: The first page load can take time. Wait for the district selector, layer controls, map container, and score panel before judging success.



Metro and bus layers rendered locally with OpenStreetMap fallback tiles.

Local Verification Gates

UI readiness

Dashboard loads

Open http://127.0.0.1:8000 and wait for the app shell.

local browser

Map renders

Azure Maps or OpenStreetMap should render with route overlays.

map + fallback

Districts appear

The selector should show 10 districts and update the panel.

district selector

API checks

Health endpoint

Return a small OK response from FastAPI.

curl -s /health

Data status

Confirm sample mode, counts, and fallback notes.

curl -s /api/data-status

Score endpoint

Call or trigger the endpoint to confirm scoring.

/api/score?districtId=...

Builder commands

Validate data

Run this when map layers or counts look wrong.

```
python scripts/validate_data.py
```

Run tests

Catch API, health, data-shape, and scoring regressions.

```
python -m pytest
```

Stay local first

Move to cloud only after the local proof path is stable.

```
DATA_MODE=sample
```

Success signal: Dashboard visible, overlays present, selector populated, score panel updating, /health OK, and /api/data-status explaining the active data source.

6. Backend Walkthrough

The backend is a small FastAPI application. It serves both the API and the static frontend so the starter kit stays easy to run and easy to deploy.

Backend API contracts

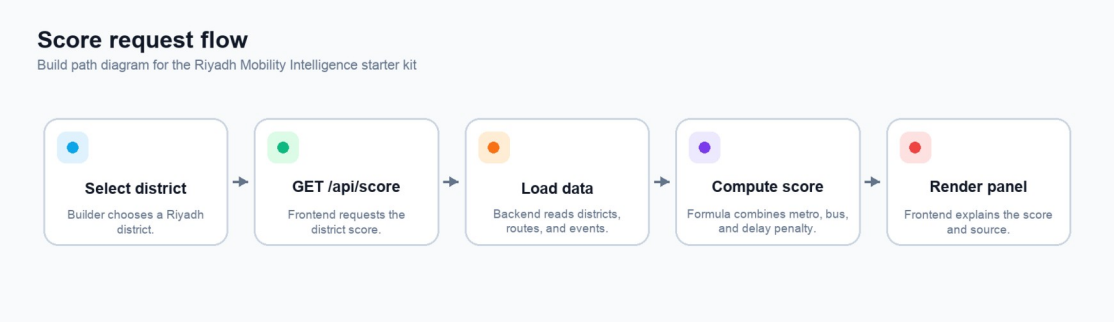
Bootstrap /health and /api/config confirm app health and runtime settings. health/config	**Map layers** /api/routes and /api/routes/geojson?mode=... return counts and map-ready geometry. routes
District scoring /api/districts and /api/score?districtId=... drive the selector and score panel. score	**Diagnostics** /api/live-events and /api/data-status explain events, mode, and fallback behavior. status

GET /api/data-status

Actual local response captured from http://127.0.0.1:8000


```
{
  "requestedMode": "sample",
  "routes": {
    "count": 106,
    "source": "sample"
  },
  "districts": {
    "source": "sample"
  },
  "liveEvents": {
    "enabled": true,
    "source": "sample",
    "count": 2
  },
  "activeMode": "sample",
  "fallbackMessage": "Azure services are optional. The app falls back to bundled sample files when
  Blob Storage or Cosmos DB is unavailable."
}
```


/api/data-status confirms sample mode and explains fallback path.



Score request flow

```
score = (nearby metro count x 3) + nearby bus count − live delay penalty
```

Adaptation note: Keep this formula easy to explain. Teams can later replace the weights or add parking, walkability, public services, heat, or event density.

7. Frontend Walkthrough

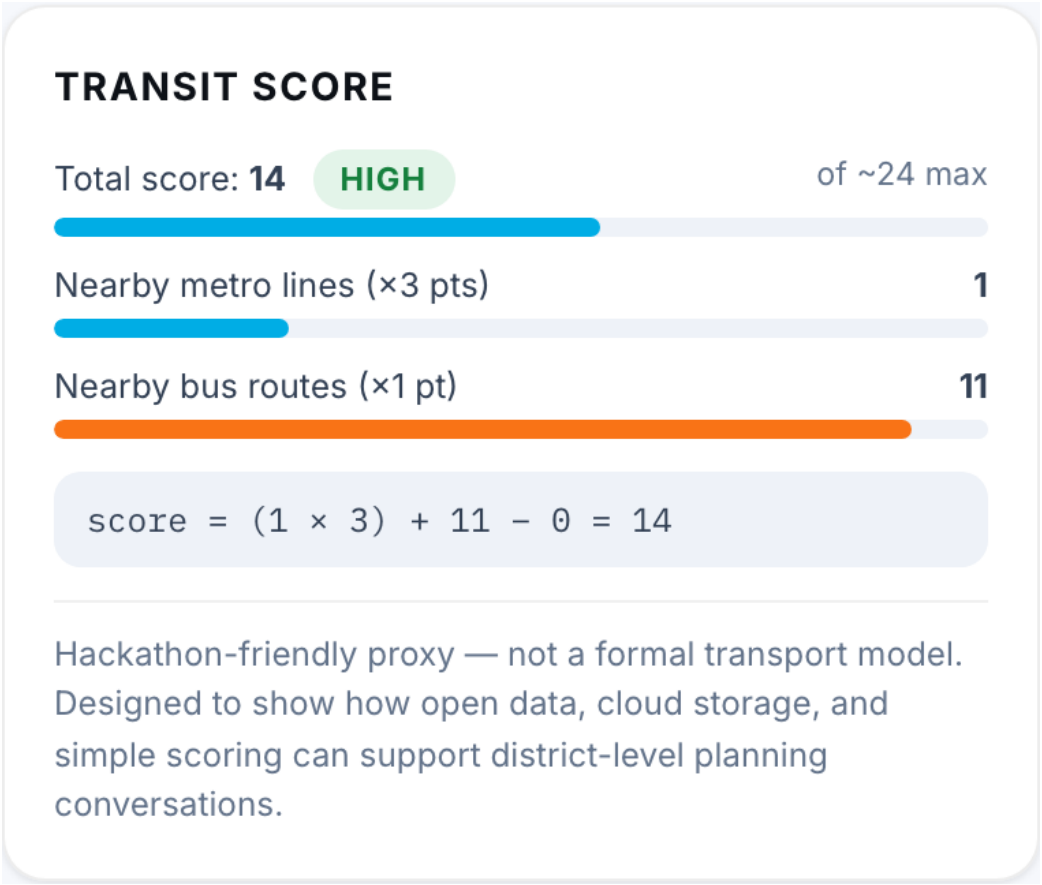
The frontend is a single-page application built with Vanilla JavaScript. On startup it fetches config, routes, metro, bus, districts, events, and data status in parallel.

Layer	What it shows	Why it matters
Metro lines	high-capacity mobility corridors	shows structural transit coverage
Bus routes	surface transit coverage	shows fine-grained network reach
Districts	selectable district points	anchors the score conversation
Live events	delay or incident markers	demonstrates cloud-live extension potential
Accessibility buffer	rough selected-district service area	makes the score formula visible

District selector in the dashboard.

Score panel story

Selected district The panel names the district in focus.	**Score and rating** A readable number and label summarize access.	**Route counts** Metro and bus counts explain the score.
Delay penalty Live event data can reduce the score.	**Formula** The current calculation is visible and replaceable.	**Data source badge** The panel explains sample, Blob, or Cosmos mode.



District score panel after selecting a Riyadh district.

8. Cloud-Live Deployment

The cloud-live path is the deployable version of the same starter kit. Use it when a team needs to show that the app can move beyond a laptop and into a credible Azure environment.

Deployment workflow

Authenticate az login and azd auth login. identity	**Initialize** azd init and set target environment if needed. environment	**Deploy** Run bash scripts/deploy_azure.sh. provision
Verify Open WEB_APP_URL, /health, and /api/data-status. smoke tests	**Operate** Use App Insights and logs if the app fails or loads slowly. diagnostics	**Teardown** Run destroy_resource_group.sh --yes when finished. cleanup

```
az login
azd auth login
azd init
bash scripts/deploy_azure.sh
bash scripts/deploy_azure.sh eastus rg-riyadh-ud-eastus
```

Azure resource	Role
Azure Container Apps	hosts FastAPI and the static frontend
Azure Container Registry	stores the built container image
Azure Maps account	supports cloud map services
Azure Blob Storage	stores raw and processed mobility files
Azure Cosmos DB	stores route, district, and event records
Application Insights	tracks health, latency, failures, and requests
Log Analytics	stores logs and diagnostics

After deploy: Copy WEB_APP_URL from azd output, open the deployed app, check /health and /api/data-status, confirm active mode, and inspect App Insights if anything fails.

9. Starter Kit Adaptation Routes

Treat this codebase as a scaffold. Keep the working shell, replace the domain data, and adapt the score or map layers toward the track the team is pursuing.

Route	Best fit	Keep / add	Likely services
Mobility Command View	Transformational Technology	Keep route layers, event overlay, selector, KPIs, data-status. Add congestion, parking demand, peak simulation, delay explanations.	Azure Maps, Event Hubs, Stream Analytics, Cosmos DB, App Insights
District Intelligence	Prosperous People	Keep district selector, score panel, map focus, Cosmos-ready records. Add service access, walkability, comparison.	Azure Maps, Blob Storage, Cosmos DB, Container Apps, optional Azure OpenAI
Sustainable Mobility Overlay	Sustainable Solutions	Keep mobility layers and selected district context. Add AQI, heat, pollution, clean corridors, emissions scoring.	Azure Maps, Blob Storage, Cosmos DB, Power BI, App Insights
Culture and Visitor Movement	Culture	Keep map layers, event markers, route overlays, selected place panel. Add heritage sites, multilingual notes, event-day access.	Azure Maps, Cosmos DB, Blob Storage, Container Apps, optional translation

Winning pattern: Preserve the local-first API, map shell, fallback chain, and deployment path. Replace the domain data and scoring logic only after the core path is working.

10. Demo Flow

Step	Show	Explain
1	Dashboard landing view	This is a Riyadh mobility intelligence starter kit, not just a static map.
2	Metro and bus toggles	The map layers show public transport coverage.
3	District selector	District context drives the scoring workflow.
4	Score panel	The score is transparent and easy to adapt.
5	/api/data-status	The app is fallback-aware and cloud-live ready.
6	Azure workflow	Services map to hosting, maps, files, records, and monitoring.
7	Adaptation routes	The same scaffold supports multiple hackathon tracks.

Demo rule: Keep the live demo local unless the deployed app has already passed /health and /api/data-status checks.

Appendix

Common Commands

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python -m uvicorn app.main:app --reload
python -m pytest
python scripts/validate_data.py
bash scripts/deploy_azure.sh
bash scripts/destroy_resource_group.sh --yes
```

Debugging Checklist

- If the page loads slowly, wait for the district selector, layer controls, map area, and score panel before judging success.
- If the map is blank, clear AZURE_MAPS_KEY in .env and reload so the local fallback can be used.
- If cloud data is missing, open /api/data-status first.
- If sample files fail, run python scripts/validate_data.py.
- If the deployed app fails, check /health, /api/data-status, Container App logs, and Application Insights.

Resource Index

Need	Open
GitHub repo	https://github.com/Esturban/microsoft-hackathon-riyadh-mobility
Local dashboard	http://127.0.0.1:8000
atomcamp Arabia	https://atomcamparabia.com/
Azure Developer CLI docs	https://learn.microsoft.com/en-us/azure/developer/azure-developer-cli/
Azure Maps docs	https://learn.microsoft.com/en-us/azure/azure-maps/
Student quickstart	README.md
Architecture notes	docs/architecture.md
Azure deployment notes	docs/azure_deployment.md
Troubleshooting	docs/troubleshooting.md